INFORME COMPLETO - FASE 2

MODELADO ORIENTADO A OBJETOS UML

Sistema: Sistema de Gestión Académica (SGA-DO)

Programa: Diplomado en Programación

Fecha de Entrega: 03 de agosto de 2026

Estudiante: Divanoska Jorelys López Zerpa

Institución: DiplomadosOnline.com

1. INTRODUCCIÓN

El presente documento tiene como objetivo presentar el diseño arquitectónico del **Sistema de Gestión Académica (SGA-DO)** mediante un diagrama de clases UML y un diagrama de flujo. Este modelado constituye la **Fase 2** del proyecto y servirá como guía fundamental para las implementaciones en Python, Java y C++.

El diseño se ha elaborado siguiendo estrictamente las especificaciones establecidas en el documento del proyecto, aplicando los principios de Programación Orientada a Objetos con énfasis en herencia y polimorfismo.

DIAGRAMA DE CLASES UML

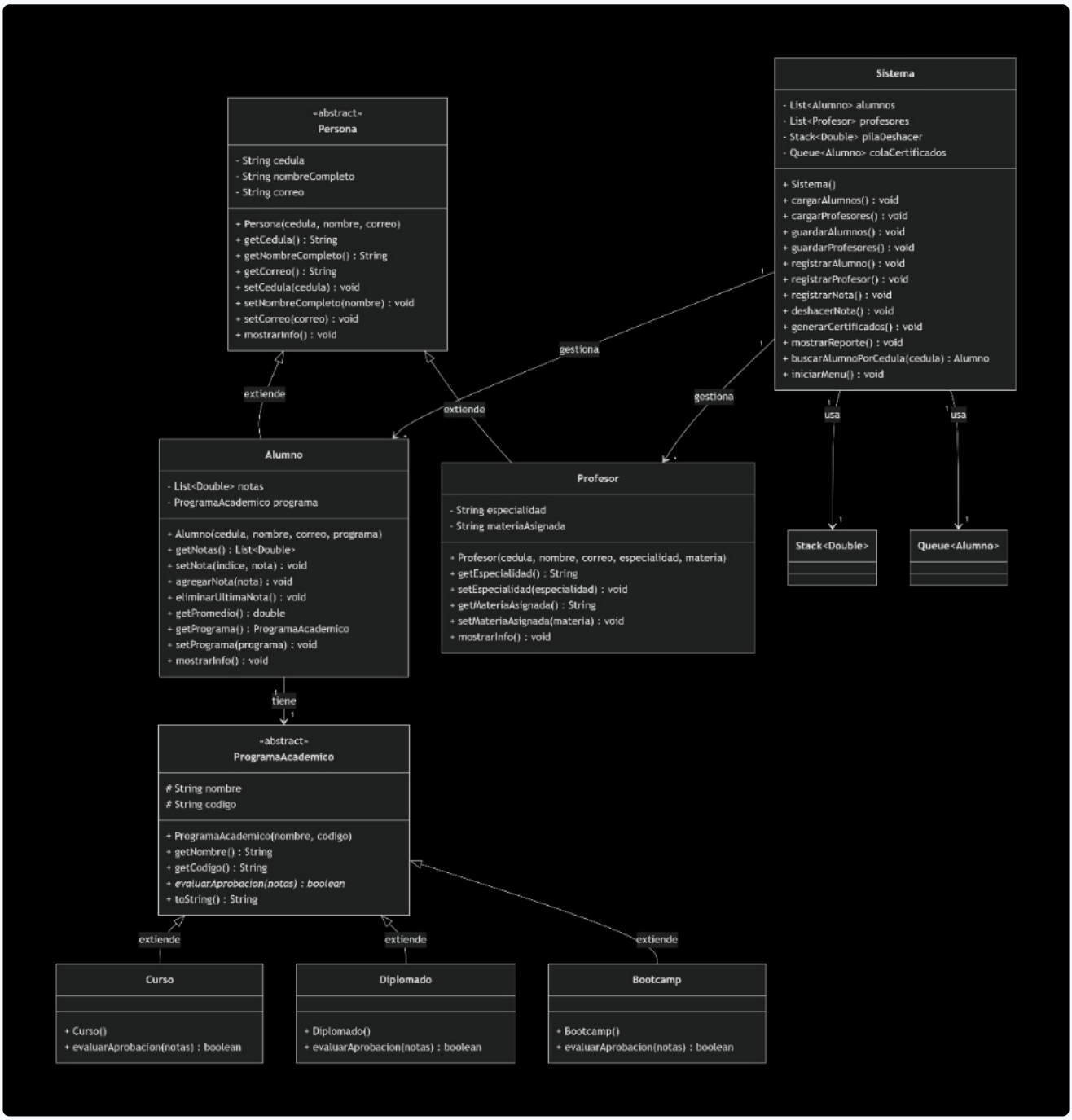


Figura 1: Diagrama de Clases UML del Sistema de Gestión Académica (SGA-DO).

2. ESTRUCTURA DEL DIAGRAMA DE CLASES

2.1 Jerarquía de Personas

La jerarquía de personas está compuesta por una clase base abstracta llamada `Persona` y dos clases derivadas: `Alumno` y `Profesor`.

2.1.1 Clase Persona (Abstracta)

La clase `Persona` es la clase base que contiene los atributos comunes a todos los individuos del sistema. Ha sido diseñada como abstracta para evitar que se instancie directamente.

Atributos:

Atributo	Tipo	Visibilidad	Descripción
`cedula`	`String`	**private**	Número de identificación único
`nombreCompleto`	`String`	**private**	Nombre completo del individuo
`correoElectronico`	`String`	**private**	Correo electrónico de contacto

Métodos:

Método	Visibilidad	Tipo Retorno	Descripción
`Persona()`	**public**	`Constructor`	Inicializa los atributos básicos
`getCedula()`	**public**	`String`	Retorna la cédula
`setCedula()`	**public**	`void`	Asigna la cédula
`getNombreCompleto()`	**public**	`String`	Retorna el nombre completo
`setNombreCompleto()`	**public**	`void`	Asigna el nombre completo
`getCorreoElectronico()`	**public**	`String`	Retorna el correo electrónico
`setCorreoElectronico()`	**public**	`void`	Asigna el correo electrónico
`mostrarInfo()`	**public**	`void`	*Método abstracto a implementar por las hijas

2.1.2 Clase Alumno (Hereda de Persona)

La clase `Alumno` representa a los estudiantes registrados en la plataforma.

Atributos adicionales:

Atributo	Tipo	Visibilidad	Descripción
notas	`List<Double>`	**private**	Lista de calificaciones (máximo 3)
programaAsignado	`ProgramaAcademico`	**private**	Programa académico al que pertenece

Métodos adicionales:

Método	Visibilidad	Tipo Retorno	Descripción
Alumno()	**public**	`Constructor`	Inicializa atributos de alumno
getNotas()	**public**	`List<Double>`	Retorna la lista de notas
setNota()	**public**	`void`	Asigna una nota en posición específica
agregarNota()	**public**	`boolean`	Agrega una nota (máximo 3)
eliminarUltimaNota()	**public**	`void`	Elimina la última nota ingresada
getPromedio()	**public**	`double`	Calcula el promedio de las notas
getProgramaAsignado()	**public**	`ProgramaAcademico`	Retorna el programa asignado
setProgramaAsignado()	**public**	`void`	Asigna el programa académico
mostrarInfo()	**public**	`void`	Sobrescribe el método de Persona

2.1.3 Clase Profesor (Hereda de Persona)

La clase `Profesor` representa a los docentes que imparten clases en la plataforma.

Atributos adicionales:

Atributo	Tipo	Visibilidad	Descripción
`especialidadAcademica`	`String`	**private**	Área de especialización (ej. Python)
`materiaAsignada`	`String`	**private**	Materia que imparte

Métodos adicionales:

Método	Visibilidad	Tipo Retorno	Descripción
`Profesor()`	**public**	`Constructor`	Inicializa atributos de profesor
`getEspecialidadAcademica()`	**public**	`String`	Retorna la especialidad
`setEspecialidadAcademica()`	**public**	`void`	Asigna la especialidad
`getMateriaAsignada()`	**public**	`String`	Retorna la materia asignada
`setMateriaAsignada()`	**public**	`void`	Asigna la materia
`mostrarInfo()`	**public**	`void`	Sobrescribe el método de Persona

2.2 Jerarquía de Programas Académicos

Esta jerarquía está compuesta por una clase base abstracta `ProgramaAcademico` y tres clases derivadas: `Curso`, `Diplomado` y `Bootcamp`. Cada una implementa su propia regla de aprobación mediante polimorfismo.

2.2.1 Clase ProgramaAcademico (Abstracta)

La clase base que define el contrato para todos los programas académicos.

Atributos:

Atributo	Tipo	Visibilidad	Descripción
`nombre`	`String`	**protected**	Nombre del programa
`codigo`	`String`	**protected**	Código identificador del programa

Métodos:

Método	Visibilidad	Tipo Retorno	Descripción
`ProgramaAcademico()`	**public**	`Constructor`	Inicializa el programa
`getNombre()`	**public**	`String`	Retorna el nombre del programa
`getCodigo()`	**public**	`String`	Retorna el código del programa
`evaluarAprobacion()`	**public**	`boolean`	*Método abstracto (polimórfico)
`toString()`	**public**	`String`	Representación en texto del programa

Subclases de ProgramaAcademico y Reglas de Aprobación:

2.2.2 Clase Curso (Hereda de ProgramaAcademico):
• **Regla de aprobación:** El promedio de las 3 notas debe ser ≥ 10/20.
• **Método:** `evaluarAprobacion(notas)` implementa la regla del Curso.

2.2.3 Clase Diplomado (Hereda de ProgramaAcademico):
• **Regla de aprobación:** El promedio de las 3 notas debe ser ≥ 14/20.
• **Método:** `evaluarAprobacion(notas)` implementa la regla del Diplomado.

2.2.4 Clase Bootcamp (Hereda de ProgramaAcademico):
• **Regla de aprobación:** Ninguna nota individual debe ser menor a 14/20 (no se evalúa promedio).
• **Método:** `evaluarAprobacion(notas)` implementa la regla del Bootcamp.

2.3 Clase Sistema (Gestor Principal)

La clase `Sistema` actúa como orquestador central de toda la aplicación.

Atributos:

Atributo	Tipo	Visibilidad	Descripción
`alumnos`	`List<Alumno>`	**private**	Lista de alumnos en memoria
`profesores`	`List<Profesor>`	**private**	Lista de profesores en memoria
`pilaDeshacer`	`Stack<Double>`	**private**	Pila LIFO para deshacer notas
`colaCertificados`	`Queue<Alumno>`	**private**	Cola FIFO para certificados

Métodos principales:

Método	Visibilidad	Tipo Retorno	Descripción
`Sistema()`	**public**	`Constructor`	Inicializa el sistema
`cargarAlumnos()`	**public**	`void`	Carga alumnos desde `alumnos.txt`
`cargarProfesores()`	**public**	`void`	Carga profesores desde `profesores.txt`
`guardarAlumnos()`	**public**	`void`	Guarda alumnos en `alumnos.txt`
`guardarProfesores()`	**public**	`void`	Guarda profesores en `profesores.txt`
`registrarAlumno()`	**public**	`void`	Flujo de registro de alumno
`registrarProfesor()`	**public**	`void`	Flujo de registro de profesor
`registrarNota()`	**public**	`void`	Flujo de registro de nota
`deshacerNota()`	**public**	`void`	Flujo de deshacer última nota
`generarCertificados()`	**public**	`void`	Flujo de generación de certificados
`mostrarReporteGeneral()`	**public**	`void`	Muestra el reporte en consola
`buscarAlumnoPorCedula()`	**public**	`Alumno`	Busca alumno por cédula
`iniciarMenu()`	**public**	`void`	Inicia el menú interactivo

3. RELACIONES ENTRE CLASES

3.1 Relaciones de Herencia

Relación	Descripción
`Persona → Alumno`	`Alumno` hereda de `Persona`
`Persona → Profesor`	`Profesor` hereda de `Persona`
`ProgramaAcademico → Curso`	`Curso` hereda de `ProgramaAcademico`
`ProgramaAcademico → Diplomado`	`Diplomado` hereda de `ProgramaAcademico`
`ProgramaAcademico → Bootcamp`	`Bootcamp` hereda de `ProgramaAcademico`

3.2 Relaciones de Asociación

Relación	Tipo	Descripción
`Alumno → ProgramaAcademico`	`1 → 1`	Cada alumno tiene un programa académico
`Sistema → Alumno`	`1 → *`	El sistema gestiona múltiples alumnos
`Sistema → Profesor`	`1 → *`	El sistema gestiona múltiples profesores
`Sistema → Stack`	`1 → 1`	El sistema utiliza una pila para deshacer
`Sistema → Queue`	`1 → 1`	El sistema utiliza una cola para certificados

4. DIAGRAMA DE FLUJO

El diagrama de flujo representa visualmente el flujo de ejecución del sistema a través de las 7 opciones del menú principal. Cada opción ha sido mapeada con sus respectivos procesos de validación, creación de objetos, persistencia en archivos y estructuras de datos.

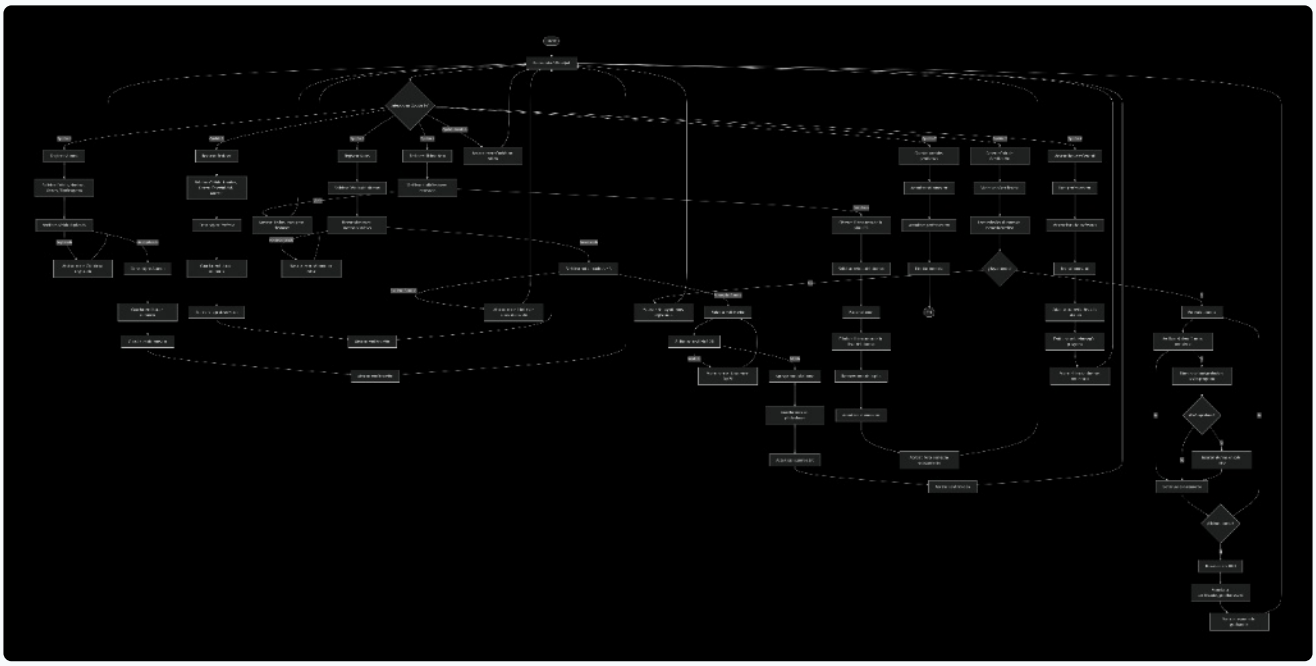


Figura 2: Diagrama de Flujo del Menú Principal y Procesos del SGA-DO.

4.1 Estructuras de Datos Implementadas

Estructura	Tipo	Propósito	Operación
Pila (Stack)	`LIFO`	Deshacer última nota	Push al agregar, Pop al deshacer
Cola (Queue)	`FIFO`	Generación de certificados	Enqueue al aprobar, Dequeue al exportar
Listas (List)	`Dram/Memoria`	Almacenamiento en memoria	Almacenar y gestionar alumnos y profesores

4.2 Persistencia en Archivos

Archivo	Formato	Propósito
`alumnos.txt`	`CSV`	Almacenar datos de alumnos (Cédula, Nombre, Correo, TipoPrograma, Nota1, Nota2, Nota3)
`profesores.txt`	`CSV`	Almacenar datos de profesores (Cédula, Nombre, Correo, Especialidad, Materia)
`certificados_pendientes.txt`	`Texto legible`	Reporte de graduandos para el departamento de RRHH

5. CONSIDERACIONES DE DISEÑO

5.1 Polimorfismo: El método `evaluarAprobacion()` en la clase `ProgramaAcademico` se declara como abstracto, forzando a las clases hijas a implementar su propia lógica. Esto permite que el sistema trate a todos los programas de manera uniforme, pero con comportamientos diferentes según el tipo.

5.2 Encapsulamiento: Todos los atributos de las clases tienen visibilidad `private` o `protected`, y se accede a ellos mediante métodos getters y setters, garantizando el principio de encapsulamiento.

5.3 Resiliencia: El sistema implementa manejo de excepciones (`try-catch`) en todas las operaciones de entrada de datos para evitar que el programa se cierre inesperadamente cuando el usuario ingresa datos incorrectos.

5.4 Gestión de Memoria (para C++): Para la implementación en C++, se debe prestar especial atención a la gestión de memoria. Cada objeto creado dinámicamente con `new` debe tener su correspondiente liberación con `delete` para evitar fugas de memoria.

6. VALIDACIÓN CON CASOS DE PRUEBA

El diseño presentado ha sido validado contra los casos de prueba establecidos en el PDF del proyecto:

Caso de Prueba	Descripción	Estado
`CP-01`	Cálculo polimórfico correcto	✅ **Diseñado para pasar**
`CP-02`	Funcionalidad LIFO (Deshacer)	✅ **Diseñado para pasar**
`CP-03`	Funcionalidad FIFO (Certificados)	✅ **Diseñado para pasar**
`EVAL-01`	Persistencia crítica (Apagón)	✅ **Diseñado para pasar**
`EVAL-02`	Polimorfismo real vs falso	✅ **Diseñado para pasar**
`EVAL-03`	Resiliencia (Try/Catch)	✅ **Diseñado para pasar**
`EVAL-04`	Auditoría de memoria (C++)	✅ **Diseñado para pasar**

7. CONCLUSIONES

El modelo UML presentado cumple con todos los requerimientos establecidos en el documento del proyecto para la Fase 2. La arquitectura diseñada se basa en los principios de Programación Orientada a Objetos, utilizando herencia y polimorfismo para garantizar un código limpio, mantenible y escalable.

Aspectos clave del diseño:

- **Jerarquía de Personas:** Permite gestionar de manera uniforme a alumnos y profesores, compartiendo atributos comunes.
- **Jerarquía de Programas Académicos:** Implementa polimorfismo para que cada tipo de programa tenga su propia regla de aprobación, facilitando la extensión futura a nuevos tipos de programas.
- **Estructuras de Datos:** Se han incluido Pila (LIFO) para la funcionalidad de deshacer y Cola (FIFO) para la generación de certificados, cumpliendo con los requerimientos funcionales.

- **Persistencia:** El diseño contempla la escritura y lectura de archivos de texto plano (.txt) para garantizar la persistencia de los datos.
- **Robustez:** El diseño incluye consideraciones de manejo de excepciones y validación de entradas para asegurar la estabilidad del sistema.

Este diseño servirá como guía fundamental para las implementaciones en Python, Java y C++ en las fases posteriores del proyecto.